TALLER DE METODOLOGÍAS ÁGILES · 2026-2 · TERCER CORTE

Punto de control del proyecto final

Programar en pareja con una persona o con un agente de IA: qué se gana, qué se pierde y qué aprendizaje desaparece

TEMA DE EXPOSICIÓN

3 · Pair programming

PRÁCTICA QUE ASISTE EL AGENTE

Desarrollo de una tarea con el agente como copiloto

CASO BASE

Notitas: un tablero kanban de notas adhesivas

PAREJA

Ana Sofía Arango Yanza · Juan José Arias Chacua

PROFESOR

César Jesús Pardo Calvache, Ph.D.

VERSIÓN

v3 · noviembre de 2026

REPOSITORIO

github.com/Sofii141/CAMEIA-lite · las referencias [R1]...[R15] de este documento están enlazadas en la sección 11

1 El proyecto en una página

Queremos saber qué cambia cuando una persona programa con un agente de IA como compañero, en lugar de programar en pareja con otra persona.

1. **La práctica.** En el *pair programming* dos personas programan juntas: una escribe el código (el *conductor*) y la otra revisa y guía (el *navegante*). Comparamos eso con una persona que programa sola con un agente de IA haciendo de navegante.
2. **El material (caso base).** Construimos **Notitas**, un tablero kanban de notas adhesivas con un tablero visual ya terminado y **24 reglas de backend sin programar**. Cada regla es una tarea pequeña, con sus criterios de aceptación y sus pruebas automáticas [R1] [R2].
3. **Antes del agente (línea base).** Programamos 10 de esas tareas en pareja, sin IA, cronometrando y grabando.
4. **Con el agente (dos ciclos de una semana).** Programamos otras tareas, cada uno solo con el agente, y registramos cada uso del agente en una bitácora.
5. **La comparación.** Medimos en las dos condiciones lo mismo: tiempo, errores que se escapan, retrabajo y cuánto entendemos del código que quedó (sección 4).

CASO BASE

24 tareas

10 de ellas difíciles a propósito

AGENTE

1

OpenCode con DeepSeek V4.1 Flash (sección 8)

LÍNEA BASE

10 tareas

en pareja y sin IA (sección 6)

Términos que usamos en el documento

Línea base	La medición de cómo trabajamos hoy, en pareja y sin IA, antes de usar el agente.
Ciclo	Una semana de trabajo real con el agente, sobre al menos 5 tareas.
Tamaño S, M, L	Tarea pequeña (unos 15–25 min), mediana (25–45 min) o grande (45–70 min), según su complejidad (sección 3).
Pruebas visibles	Pruebas automáticas que se ven mientras se programa y dicen cuándo la tarea está terminada.
Pruebas ocultas	Pruebas que nadie ve hasta terminar. Cada una que falla es un error que se escapó.
Codewalk	Cinco preguntas sobre el código, respondidas sin mirarlo, para medir cuánto se entendió (0 a 5).
Bitácora	La hoja de cálculo donde se anota cada uso del agente y si se aceptó, se corrigió o se rechazó, y por qué.

2 Qué esperamos ver

La literatura revisada por pares sobre programar con IA apunta a tres cosas, y cada una corresponde a una de nuestras mediciones:

Qué esperamos	Por qué lo esperamos
Que el tiempo baje en las tareas sencillas, pero no en las difíciles	Con IA los estudiantes terminaron un 35 % más rápido (Shihab et al., 2025), pero el efecto de programar en pareja depende de la complejidad de la tarea (Hannay et al., 2009; Arisholm et al., 2007).
Que se escapen más errores y haya más retrabajo	Las sugerencias de la IA se aceptan con menos escrutinio que las de un compañero (Welter et al., 2025), y con IA se escribe código menos seguro creyendo que es más seguro (Perry et al., 2023).
Que se entienda menos el código, sobre todo el compañero que no participó	En pareja se transfiere conocimiento (Plonka et al., 2015); con IA, los estudiantes no entendían por qué funcionaban las sugerencias (Shihab et al., 2025). Aunque otro estudio no encontró pérdida de aprendizaje (Kazemitabaar et al., 2023).

Son expectativas, no conclusiones: si los datos dicen otra cosa, lo reportaremos así. Con unas 10 tareas por condición no podremos afirmar causas, solo tendencias (sección 7).

3 El caso base: Notitas

Notitas es un tablero de notas adhesivas con tres columnas (*Por hacer · En curso · Hecho*). El tablero visual ya está hecho; lo que falta son las reglas que lo hacen funcionar.

Cómo está hecho

El repositorio tiene dos partes [R1]. El **frontend** [R7] es el tablero, con arrastrar y soltar y animaciones; ya está terminado y **no se mide**. El **backend** [R3] tiene 24 funciones vacías: cada tarea consiste en escribir la lógica de una de ellas. En cuanto una función funciona, el tablero empieza a usarla.

Así, el cronómetro mide solo el desarrollo de la tarea, no la configuración, y todas las tareas se pueden comparar entre sí. Ninguno de los dos había programado antes estas reglas.

Qué lo hace representativo y difícil

- Son las reglas típicas de un tablero real: validar datos, mover notas entre columnas, buscar y filtrar, fechas de vencimiento, deshacer, exportar.
- **10 de las 24 son difíciles a propósito**: concurrencia, zonas horarias, texto con tildes y otros caracteres, inyección de código en el contenido, copias que comparten datos sin querer y reglas que parecen obvias pero no lo son.
- Cada tarea tiene pruebas visibles y ocultas [R4]: **41 visibles** y **72 ocultas**. Comprobamos que las 113 fallan con las funciones vacías y pasan con soluciones de referencia, que no están en el repositorio.

Tamaño de las tareas y en qué se basa

Tamaño	Cuántas	Tiempo estimado	Criterio
pequeña (S)	10	15–25 min	Una función que valida o transforma un dato, con reglas independientes entre sí
mediana (M)	12	25–45 min	Varias reglas que interactúan, fechas o zonas horarias, o una clase que guarda estado
grande (L)	2	45–70 min	Concurrencia o un algoritmo de varios pasos con casos de conflicto

Es una estimación por juicio de quien diseñó las tareas, revisada por la pareja antes de congelar el caso. No es un dato del estudio: solo sirve para que cada condición reciba una mezcla parecida de tareas. El número de criterios de aceptación no distingue los tamaños (todas tienen entre 3 y 5), por eso no lo usamos. La duración real la da el cronómetro, y si un tamaño no se cumple, lo reportaremos.

Las 24 tareas [R2]

ID	Tarea	Tamaño	Por qué es difícil
`t01`	Validar el título de una nota	pequeña	

ID	Tarea	Tamaño	Por qué es difícil
t02	Color de la nota	pequeña	
t03	Transiciones entre columnas	mediana	Difícil Muchas combinaciones y una regla que parece obvia pero no lo es (no se salta "En curso")
t04	Límite WIP de "En curso"	mediana	Difícil Reordenar dentro de la misma columna no debe contar como una nota más
t05	Insertar en una posición	mediana	
t06	Filtrar por etiquetas	pequeña	
t07	Buscar sin tildes ni mayúsculas	mediana	Difícil Unicode: tildes, ñ y letras como ß que no se resuelven con lower()
t08	Estado de vencimiento	mediana	Difícil Zona horaria: a las 10 p. m. de Bogotá ya es el día siguiente en UTC
t09	Normalizar etiquetas	pequeña	
t10	Qué notas se archivan	pequeña	
t11	Deshacer movimientos	mediana	
t12	Dos personas mueven la misma nota	grande	Difícil Concurrencia: dos llamadas al mismo tiempo con la misma versión; solo una puede ganar
t13	Exportar e importar el tablero	mediana	
t14	Contenido seguro (sin inyectar HTML)	mediana	Difícil Trampa de seguridad: hay que escapar antes de dar formato, no después
t15	Resumen por columna	pequeña	
t16	Ordenar por prioridad y fecha	pequeña	
t17	Paginar el listado	pequeña	
t18	Iniciales del responsable	pequeña	
t19	Progreso de la checklist	mediana	
t20	Duplicar una nota	pequeña	Difícil Trampa de mutabilidad: una copia superficial comparte las listas con la original
t21	Fusionar cambios de dos dispositivos	grande	Difícil Fusión a tres bandas, empates y fechas con zonas horarias distintas
t22	Notas bloqueadas	mediana	

ID	Tarea	Tamaño	Por qué es difícil
`t23`	Tiempo de ciclo	mediana	Difícil La nota puede ir y volver entre columnas, y los eventos pueden llegar desordenados
`t24`	Límite de notas por minuto	mediana	Difícil Ventana deslizante: los bordes y los intentos rechazados

LIMITACIONES QUE DECLARAMOS

- **Es un caso diseñado para el experimento**, no el backlog de un proyecto en curso: los resultados no se generalizan a un sistema real grande.
- Las tareas, las pruebas y el tablero se generaron con Claude Code. Para que el caso no quede "a la medida" del agente: el agente es **otro modelo** (DeepSeek), nadie ve las pruebas ocultas antes de terminar, el agente no tiene acceso a ellas y se incorporan 3 casos de otra pareja.
- Las tareas son reglas aisladas: no medimos el costo de integrarlas con un sistema más grande.

4 Qué medimos (GQM)

Objetivo: analizar el desarrollo de una tarea con un agente copiloto, frente al pair programming humano, para saber si cambian el tiempo, los errores que se escapan, el retrabajo y la comprensión del código.

Son cinco métricas **directas** [R10]: cada una es un número que se anota tal cual, sin fórmulas. Las tres primeras son las que el enunciado sugiere para el tema 3 (tiempo, retrabajo y errores detectados a tiempo); la cuarta responde la pregunta del tema (qué aprendizaje desaparece) y la quinta muestra si la sensación de velocidad engaña.

Pregunta	Métrica	Cómo se toma	Qué NO representa
¿Se termina más rápido?	**M1** Minutos hasta que pasan las pruebas visibles	Hora de inicio y de fin, menos pausas	La productividad en general
¿Se detectan los errores a tiempo?	**M2** Pruebas ocultas que fallan al decir "terminado"	Se corren las ocultas de esa tarea	Defectos de diseño o de legibilidad
¿Hay retrabajo?	**M3** Minutos de corrección después de "terminado"	Cronómetro aparte (0 si no hubo)	El tamaño de la corrección
¿Qué aprendizaje se pierde?	**M4** Codewalk de 0 a 5, al autor y al compañero	5 preguntas sin mirar el código [R11]	La habilidad general de la persona
¿Nos engaña la sensación de velocidad?	**M5** Pronóstico en minutos, antes de empezar	Se compara con M1	El resultado real

Las 5 preguntas del codewalk

Son las mismas para todas las tareas, porque preguntan por la comprensión y no por una tarea en particular. Solo cambian la X y la Y, que están fijadas por tarea para que sean iguales en las dos condiciones [R11].

1. ¿Qué hace la función, en una frase?
2. ¿Qué caso borde maneja y dónde?
3. ¿Por qué se eligió esta forma y no otra?
4. Si mañana falla con la entrada X, ¿dónde mirarías primero?
5. ¿Qué cambiarías si el requisito cambia en Y?

En los ciclos, la bitácora registra además cada uso del agente con su decisión (aceptar, corregir o rechazar), su razón y su responsable [R9].

5 Cómo medimos

	Línea base	Ciclos 1 y 2
Quién trabaja	Los dos juntos, sin IA: uno escribe y el otro guía, y cambian cada 15 min	Una persona sola con el agente; el compañero no participa en esa tarea
Cuántas tareas	10: 5 pequeñas, 4 medianas y 1 grande	12 (6 por ciclo): 5 pequeñas, 6 medianas y 1 grande; cada uno conduce 6. En el ciclo 2 se suman los 3 casos de otra pareja
Evidencia	Grabación de pantalla, un commit al empezar, al terminar y al corregir cada tarea, la hoja de medición y la hoja de sesiones [R9]; en los ciclos, además, una fila de bitácora por cada uso del agente	

Pasos por tarea

1. Anotar cuántos minutos creemos que tomará (M5).
2. Abrir el enunciado e iniciar el cronómetro.
3. Programar con ayuda solo de las pruebas visibles.
4. Cuando pasan, declarar "terminado" y parar el cronómetro (M1).
5. Correr las pruebas ocultas y anotar cuántas fallan (M2). Si corregimos, ese tiempo es retrabajo (M3).
6. Codewalk al autor y al compañero (M4).
7. Llenar la hoja de medición en ese momento, no al final del día.

CONTROLES PARA QUE LA COMPARACIÓN SEA JUSTA

- **Sorteo** [R5] [R6]: un programa reparte las tareas entre la línea base y los ciclos, de modo que cada condición reciba una mezcla parecida de tamaños y de tareas difíciles; también sortea el orden. Se puede repetir con la misma semilla y da lo mismo.
- **Caso congelado** antes de medir: desde ese momento no se cambian las tareas ni las pruebas.
- Si alguien ve las pruebas ocultas antes de tiempo, esa tarea se descarta y se usa una de repuesto.
- El agente trabaja en una copia del proyecto **sin** las pruebas ocultas y con permisos limitados [R8].

6 Línea base

Lo que sigue lo genera un script a partir de la bitácora **[R9]** **[R15]**, con lo medido en las sesiones cronometradas y grabadas. No se escribe a mano.

PENDIENTE DE MEDIR

Esta sección la llena `07 - Analisis/Scripts/resumen_medicion.py` a partir de la bitácora, después de cada sesión.

7 Cómo presentaremos los resultados

Con pocas tareas por condición, mostramos todos los datos y decimos qué no demuestran [R14].

Qué reportamos	Ejemplo	Fuente
Mediana [mínimo–máximo] de minutos y del codewalk, por tamaño	"Tareas medianas: 32 min [21–48] en pareja y 25 min [18–60] con el agente"	Kitchenham et al., 2017
Conteos de errores escapados y de retrabajo	"Se escaparon 4 de 30 errores; 3 de 10 tareas tuvieron retrabajo"	Arisholm et al., 2007
Pronóstico frente a tiempo real	"En 7 de 12 tareas tardamos más de lo que creíamos"	Perry et al., 2023
Decisiones de la bitácora	"De 48 usos: 20 aceptados, 21 corregidos y 7 rechazados"	Enunciado del curso
Sin afirmar causas, y en cada cifra, qué no demuestra	"No prueba que el agente cause la diferencia: son pocas tareas y distintas en cada condición"	Kitchenham et al., 2002; Wohlin et al., 2012
Una cifra recalculada por la otra persona	El script, la hoja de cálculo y un cálculo a mano deben coincidir	—

8 Especificación del agente • v1 [R12]

Los 8 campos son los que pide el enunciado del proyecto. La columna *Fundamento* dice en qué investigación o norma se apoya cada decisión.

Campo	Contenido	Fundamento
Rol y alcance	Hace de navegante en el desarrollo de *una* tarea de Notitas: propone un enfoque, el código y los casos borde que conviene probar. La persona escribe y decide.	Roles de conductor y navegante (Williams et al., 2000; Bryant et al., 2008)
Entradas	Solo el enunciado de la tarea, su archivo del backend y sus pruebas visibles.	Compartir solo lo necesario (NIST AI 600-1, 2024)
Salidas	En formato fijo: un plan de 3 a 6 pasos, el cambio propuesto en el código, los supuestos que hizo y los casos borde que cree que faltan.	El enunciado: sin formato definido, una salida no se puede evaluar
Herramientas y permisos	**OpenCode** con el modelo gratuito **DeepSeek V4.1 Flash**, el mismo para los dos todo el ciclo (el modelo y la versión se anotan). Trabaja en una copia del proyecto sin las pruebas ocultas; tiene prohibido leerlas, cambiar las pruebas, guardar o subir cambios al repositorio, salir de la carpeta y buscar en internet, y para todo lo demás pide permiso [R8].	Mínimo privilegio (NIST AI 600-1, 2024). No es de la familia Claude, con la que se construyó el caso
Prohibiciones	No guardar ni subir cambios al repositorio. No modificar pruebas ni otros archivos. No leer las pruebas ocultas. No desactivar ni saltarse pruebas. No inventar requisitos. No manejar credenciales ni datos personales.	Validez de la medición (Ko et al., 2015); código inseguro con IA (Perry et al., 2023)
Criterio de validación	Se acepta solo si pasan las pruebas visibles, cumple cada criterio de aceptación y quien conduce puede explicar cada línea sin mirar al agente.	Menos escrutinio con IA (Welter et al., 2025); no entender las sugerencias (Shihab et al., 2025)
Punto de control humano	Antes de guardar cada cambio, quien conduce decide aceptar, corregir o rechazar y lo anota en la bitácora con su razón. Las pruebas ocultas las corre la persona. Después viene el codewalk.	Human-in-the-loop (NIST AI 600-1, 2024; sesión 3 del curso)
Versión	v1. Cada cambio da una versión nueva, con el dato de la bitácora que la motivó.	El enunciado

Empezamos con un solo agente. Solo agregaremos un segundo ("revisor") en el ciclo 2 si la bitácora del ciclo 1 muestra un problema que el primero no resuelve.

9 Calendario

Caso base	Línea base	Ciclo 1	Ciclo 2	16–18 nov
Congelado y sorteado	10 tareas en pareja, sin IA	Con el agente, bitácora y spec v2	3 casos de otra pareja, 3 fallos y video	Reporte, diapositivas y exposición

10 Fuentes

Todas revisadas por pares, salvo la norma de NIST. Los DOI se verificaron en Crossref y OpenAlex. La lista completa, con la base de datos de cada una, está en **[R13]**.

- Arisholm, E., Gallis, H., Dybå, T., & Sjøberg, D. I. K. (2007). Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE Transactions on Software Engineering, 33*(2), 65–86. doi:10.1109/TSE.2007.17
- Basili, V., Caldiera, G., & Rombach, H. D. (1994). The Goal Question Metric approach. En *Encyclopedia of Software Engineering*. Wiley.
- Bryant, S., Romero, P., & du Boulay, B. (2008). Pair programming and the mysterious role of the navigator. *International Journal of Human-Computer Studies, 66*(7), 519–529. doi:10.1016/j.ijhcs.2007.03.005
- Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance. *International Journal of STEM Education, 12*, 16. doi:10.1186/s40594-025-00537-3
- Hannay, J. E., Dybå, T., Arisholm, E., & Sjøberg, D. I. K. (2009). The effectiveness of pair programming: A meta-analysis. *Information and Software Technology, 51*(7), 1110–1122. doi:10.1016/j.infsof.2009.02.001
- Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. *CHI 2023*, 1–23. doi:10.1145/3544548.3580919
- Kitchenham, B. et al. (2002). Preliminary guidelines for empirical research in software engineering. *IEEE TSE, 28*(8), 721–734. doi:10.1109/TSE.2002.1027796
- Kitchenham, B., Madeyski, L., Budgen, D., Keung, J., Brereton, P., Charters, S., Gibbs, S., & Pohthong, A. (2017). Robust statistical methods for empirical software engineering. *Empirical Software Engineering, 22*(2), 579–630. doi:10.1007/s10664-016-9437-5
- Ko, A. J., LaToza, T. D., & Burnett, M. M. (2015). A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering, 20*(1), 110–141. doi:10.1007/s10664-013-9279-3
- Lee, H.-P. et al. (2025). The impact of generative AI on critical thinking. *CHI 2025*, 1–22. doi:10.1145/3706598.3713778
- Moradi Dakhel, A. et al. (2023). GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software, 203*, 111734. doi:10.1016/j.jss.2023.111734
- NIST (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile* (NIST AI 600-1). doi:10.6028/NIST.AI.600-1
- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? *ACM CCS 2023*, 2785–2799. doi:10.1145/3576915.3623157
- Plonka, L., Sharp, H., van der Linden, J., & Dittrich, Y. (2015). Knowledge transfer in pair programming: An in-depth analysis. *International Journal of Human-Computer Studies, 73*, 66–78. doi:10.1016/j.ijhcs.2014.09.001
- Shihab, M. I. H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., & Mulanda, B. (2025). The effects of GitHub Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. *ICER 2025*, 407–420. doi:10.1145/3702652.3744219
- Vargha, A., & Delaney, H. D. (2000). A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics, 25*(2), 101–132. doi:10.3102/10769986025002101
- Welter, A., Schneider, N., Dick, T., Weis, K., Tinnes, C., & Wyrich, M. (2025). An empirical study of knowledge transfer in AI pair programming. *ASE 2025*, 166–177. doi:10.1109/ASE63991.2025.00022

- Williams, L., Kessler, R. R., Cunningham, W., & Jeffries, R. (2000). Strengthening the case for pair programming. *IEEE Software, 17*(4), 19–25. doi:10.1109/52.854064
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. doi:10.1007/978-3-642-29044-2

11 Dónde está cada cosa en el repositorio

Las referencias [R1]…[R15] del texto enlazan aquí. No adjuntamos archivos aparte: todo está en el repositorio, con su historial de versiones. El código está en la rama `main` y los documentos en la rama `documentacion`.

Ref.	Qué es	Enlace
R1	Repositorio del proyecto (código del caso base)	rama main
R2	Enunciado y criterios de aceptación de las 24 tareas	TAREAS.md
R3	Código de las tareas (donde se programa la lógica)	backend/notitas/
R4	Pruebas visibles y ocultas	backend/tests/
R5	Sorteo de las tareas	sorteo.py
R6	Resultado del sorteo y registro de sesiones	registro/
R7	Tablero (frontend, no se mide) y servidor	frontend/ y backend/servidor.py
R8	Configuración y permisos del agente	agente/opencode.json
R9	Bitácora y hoja de medición	31102026_01_Bitacora_v3.xlsx
R10	GQM	31102026_01_GQM_v3.md
R11	Preguntas del codewalk con su X y su Y por tarea	31102026_02_Codewalk_Preguntas_v1.md
R12	Especificación del agente v1 (con fundamento e historial)	31102026_01_Spec_Copiloto_v1.md
R13	Fuentes revisadas por pares, con DOI	29102026_01_Fuentes_v2.md
R14	Cómo medimos y cómo presentaremos los resultados	31102026_01_Como_Medir_y_Presentar_v3.md
R15	Script que resume la bitácora	resumen_medicion.py

12 Declaración de uso de IA

Herramienta	Para qué	Qué verificamos nosotros
Claude Code	Leer los enunciados; proponer la estructura; generar las 24 tareas con sus pruebas y soluciones de referencia, el backend y el tablero; buscar fuentes y verificar sus DOI en Crossref y OpenAlex; revisar la coherencia de los documentos con un segundo agente revisor; maquetar este documento.	**Completar antes de entregar**: que revisamos las 24 tareas antes de congelar y qué fuentes leímos.

Ningún dato de línea base ni de bitácora fue generado por IA: esos datos salen solo de nuestras sesiones cronometradas y grabadas.